Ciência de Dados e Aprendizagem de Máquina



Cruzeiro do Sul Virtual
Educação a distância

Análise Preditiva (Parte I)

Responsável pelo Conteúdo:
Prof. Dr. Luciano Rossi

Revisão Textual:
Prof. Me. Claudio Brites

UNIDADE Análise Preditiva (Parte I)

- Conceito de Classificação;
- O Algoritmo *k-NN*;
- Árvores de Decisão;
- O Algoritmo *Naive Bayes*;
- Aplicações com a Linguagem R.



OBJETIVO DE APRENDIZADO

- Analisar as principais características da tarefa de classificação, por meio da definição de alguns dos seus principais algoritmos com a respectiva de aplicação prática.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Conceito de Classificação

Existem muitos conjuntos de dados que apresentam uma associação entre características (atributos preditivos) e a respectiva classe (atributo de classe). Por exemplo, podemos ter um conjunto de prontuários de pacientes com o registro dos sintomas e do diagnóstico de cada paciente. Esse tipo de exemplo pode ser ampliado para diversos outros contextos. Suponha que, com base em um conjunto de dados de diagnósticos médicos, queremos saber se um novo paciente é ou não cardiopata, de acordo com os seus sintomas.

A tarefa descrita anteriormente é uma tarefa de predição, na qual queremos prever uma classe (diagnóstico do paciente) de acordo com um conjunto de características (sintomas do paciente). Quando temos uma predição discreta, na qual estamos interessados em uma ou mais classes, dizemos que a tarefa é de classificação. Por outro lado, quando a predição é contínua, e estamos interessados em predizer um determinado valor, dizemos que a predição é denominada estimação.

O processo de predição apresenta duas etapas principais. Na primeira etapa, denominada treinamento, utilizamos uma parte dos dados de um conjunto (a maior parte deles) para gerar um modelo preditor que seja capaz de inferir uma classe ou um valor a partir das características dos dados. A segunda etapa, denominada teste, consiste em aplicar o modelo obtido à parte não utilizada dos dados. O objetivo aqui é verificar a capacidade de generalização do modelo, no sentido de predizer a classe ou o valor de dados que não foi utilizado para a obtenção do preditor.

A classificação é uma tarefa preditiva que tem por objetivo aprender uma função ou modelo que possa ser útil para associar os atributos descritivos a um atributo de classe. Por exemplo, a Figura 1A, apresenta um grupo de registros que representam pessoas. Cada registro está rotulado como saudável ou doente e, além disso, temos um único atributo temperatura, que também está associado aos registros. Veja que os registros são distribuídos ao longo do único eixo que descreve os valores do atributo temperatura, e os rótulos (doente ou saudável) estão representados nas cores verde e vermelho, respectivamente.

Podemos inserir um pouco mais de complexidade a partir da utilização de outros atributos descritivos. Veja que, na Figura 1B, temos um atributo adicional que se refere ao número de batimentos cardíacos da pessoa. Além disso, o padrão da distribuição dos registros no plano cartesiano não é evidente.

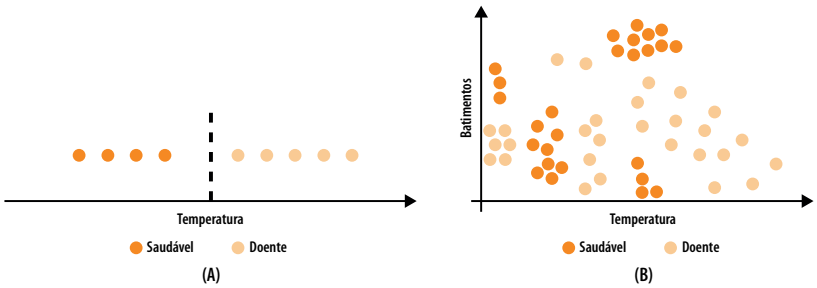


Figura 1 – Exemplo ilustrativo de classificação de objetos a partir de um e dois atributos descritivos

Note que, para a distribuição na Figura 1B, não é possível determinar um único limiar de temperatura, ou de batimentos cardíacos, que separe os objetos em grupos homogêneos. A tarefa de classificação pode demandar outras estratégias para a obtenção de um padrão que possa ser utilizado para a classificação de novos registros não rotulados. Além disso, a utilização de apenas dois atributos, no caso temperatura e batimentos cardíacos, pode não resultar em um bom preditor. No entanto, dados multidimensionais podem ser difíceis de serem explorados, assim, métodos mais sofisticados são necessários para que possamos realizar a tarefa de classificação.

O Algoritmo *k-NN*

Definições

O algoritmo *k-NN*, que significa *k* vizinhos mais próximos, classifica um objeto não rotulado por meio da comparação desse objeto com outros objetos que possuem rótulos, ou seja, o conjunto de dados de treinamento é utilizado para determinar a classe do objeto não rotulado.



Conheça mais detalhes sobre o algoritmo *k-NN* no vídeo a seguir.
Disponível em: https://youtu.be/DeAuVrhKw58

A lógica de funcionamento do *k-NN* é simples. Dado um objeto não rotulado, é verificada a classe dos vizinhos mais próximos a esse objeto e a classe majoritária da vizinhança é atribuída ao objeto sem rótulo. A identificação dos vizinhos mais próximos é feita a partir de uma métrica de distância.

As métricas de distância são utilizadas para medir o grau de similaridade ou de dissimilaridade entre dois objetos do conjunto de dados. Assim, os atributos do objeto são utilizados como dimensões no espaço dimensional. As etapas do algoritmo *k-NN* são descritas a seguir.

Algoritmo 1: *k-NN*
Entrada:
X_{tr}: conjunto de treinamento;
X_{te}: conjunto de teste;
k: número de vizinhos considerados para comparação;
d: medida de distância.
Saída:
y_{te}: rótulo estimado para o objeto de teste.
Etapas:
1. Para cada objeto $\vec{x}_i \in X_{tr}$:
2. Calcule a distância d entre $\vec{x}_i$ e $\vec{x}_{te}$;
3. Ordene os objetos, em ordem crescente, de acordo com d;
4. Selecione os k primeiros objetos em X_{tr};
5. Atribua a $\vec{x}_{te}$ o rótulo majoritário y_{te} dos k objetos de X_{tr}.

Fonte: Adaptado de SILVA *et al.*, 2017

Para ilustra a aplicação do algoritmo *k-NN*, considere o conjunto de dados apresentados na Tabela 1. Cada objeto no conjunto de dados é representado por uma linha na tabela, são 12 objetos no total. As colunas representam os atributos descritivos (tempo e preço) e o atributo de classe. Os atributos descritivos foram, previamente, normalizados no intervalo [0-1] e o atributo de classe pode ter dois valores: satisfeito (SAT) ou insatisfeito (INS).

O conjunto de dados representa o tempo que um cliente esperou para ser atendido em um restaurante e o preço que foi pago pelo cliente pelo prato escolhido. Além disso, cada cliente declarou sua satisfação ou insatisfação com o restaurante.

O conjunto de dados da Tabela 1 será considerado para a aplicação do algoritmo *k-NN*, com a utilização do *RStudio*.

Tabela 1 – Conjunto de dados de exemplo para o algoritmo *k-NN*

		x_{i1}	x_{i2}	y_i
	ID	Tempo	Preço	Classe
$\vec{x}_1$	01	0,70	0,40	SAT
$\vec{x}_2$	02	0,40	0,50	SAT
$\vec{x}_3$	03	0,61	0,40	SAT
$\vec{x}_4$	04	0,90	0,70	SAT
$\vec{x}_5$	05	0,80	0,30	SAT
$\vec{x}_6$	06	0,60	1,00	SAT
$\vec{x}_7$	07	0,20	0,60	INS
$\vec{x}_8$	08	0,40	0,30	INS
$\vec{x}_9$	09	0,61	0,20	INS
$\vec{x}_{10}$	10	0,60	0,43	INS
$\vec{x}_{11}$	11	0,50	0,10	INS
$\vec{x}_{12}$	12	1,00	0,20	INS

Fonte: Adaptada de SILVA *et al.*, 2017

Aplicações com a Linguagem *R*

A linguagem de programação *R*, ou mais especificamente o *RStudio*, disponibiliza um amplo conjunto de pacotes de funções para diferentes aplicações. O pacote ***class*** contém a função `knn()` que implementa o algoritmo *k-NN*. A aplicação desta função é descrita pelo autor da seguinte forma:

> para cada linha do conjunto de teste, os *k* vetores de conjunto de treinamento mais próximos (em distância euclidiana) são encontrados, e a classificação é decidida por maioria de votos, com empates desfeitos aleatoriamente. Se houver empate no *k*-ésimo vetor mais próximo, todos os candidatos serão incluídos na votação. (SILVA *et al.*, 2017)

A utilização da função `knn()` é feita por meio da seguinte notação:

```
knn(train, test, cl, k = 1, l = 0, prob = FALSE, use.all = TRUE)
```

em que:

- **`train`:** matriz ou quadro de dados de casos de conjunto de treinamento;
- **`test`:** matriz ou quadro de dados de casos de conjunto de teste. Um vetor será interpretado como um vetor linha para um único caso;
- **`cl`:** fator de classificações verdadeiras do conjunto de treinamento;
- **`k`:** número de vizinhos considerados;
- **`l`:** voto mínimo para decisão definitiva, caso contrário, dúvida;
- **`prob`:** se isso for verdade, a proporção dos votos para a classe vencedora é retornada com prob do atributo;
- **`use.all`:** controla o manuseio de laços. Se verdadeiro, todas as distâncias iguais ao *k*-ésimo maior serão incluídas. Se falso, uma seleção aleatória de distâncias iguais ao *k*-ésimo é escolhida para usar exatamente *k* vizinhos.

Vamos considerar os dados da Tabela 1 em um formato *CSV* (*Comma Separated Values*), que é um arquivo texto no qual os valores são separados por vírgula, de modo a facilitar a leitura por programas de computador. No *RStudio*, a aba *Environment* disponibiliza a ferramenta *Import Dataset* que pode ser utilizada para importar o arquivo CSV do computador para o *RStudio*. O resultado dessa importação é apresentado na Figura 2.

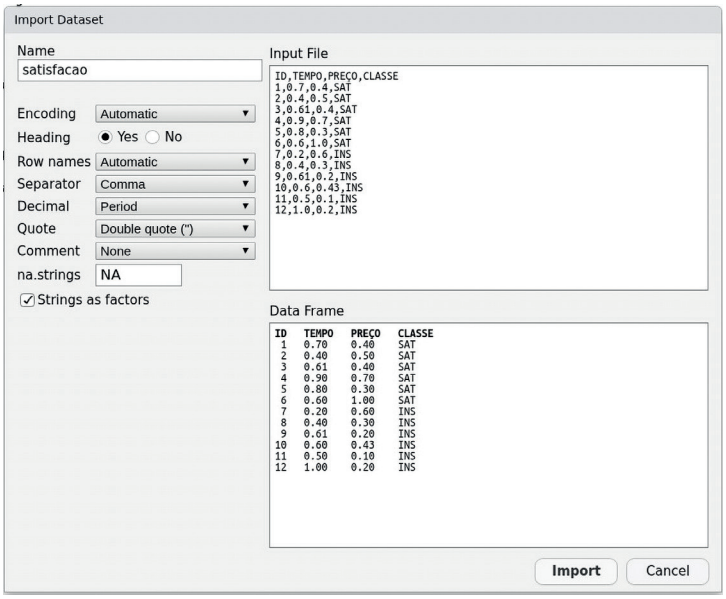


Figura 2 – Resultado da importação de um arquivo CSV (*Input File*) para o *RStudio*, transformando os dados em um *Data Frame*
Fonte: Reprodução

Conforme apresentado na Figura 2, os dados foram importados para o *RStudio* como um objeto (*Data Frame*) cujo identificador é "`satisfacao`". Podemos verificar a distribuição dos registros por meio da utilização da função `plot()`. Nesse caso, queremos passar para a função os valores das linhas 1 até 12 e das colunas 2 até 3, da seguinte maneira:

```
plot(satisfacao[1:12,2:3]).
```

O resultado do uso da função `plot()` é apresentado na Figura 3.

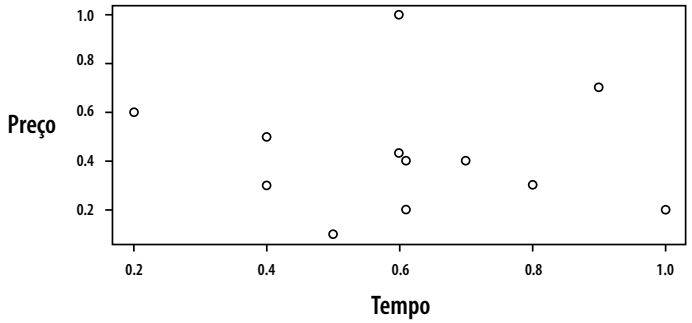


Figura 3 – Distribuição dos dados em duas dimensões, a partir da utilização da função plot()

Antes da aplicação da função `knn()`, é preciso separar os dados de treinamento e os dados para teste, no nosso caso, vamos utilizar somente um registro para teste e os demais para treinamento. Assim, vamos criar um novo objeto chamado `treinamento` e atribuir a esse objeto os dados de treinamento:

```
treinamento <- satisfacao[2:12,2:3].
```

De maneira similar, o registro que será utilizado para teste será separado em um novo objeto:

```
teste <- satisfacao[1:1,2:3].
```

Finalmente, vamos separar os rótulos dos dados em um novo objeto:

```
rotulos <- satisfacao[2:12,4:4].
```

Com os dados preparados, podemos utilizar a função `knn()` da seguinte forma:

```
y_estimado <- knn(treinamento, teste, rotulos, 3),
```

em que `y_estimado` é o rótulo predito para o registro do conjunto de teste, e os parâmetros passados para a função `knn()` são os objetos anteriormente construídos, e o valor 3 é o número de vizinhos considerados. Este pequeno experimento resulta na classificação correta do registro de teste (SAT), conforme apresentado na Figura 4.

```
> treinamento <- satisfacao[2:12,2:3]
> rotulos <- satisfacao[2:12,4:4]
> teste <- satisfacao[1:1,2:3]
> y_estimado <- knn(treinamento,teste,rotulos,3)
> y_estimado
[1] SAT
Levels: INS SAT
```

Figura 4 – Exemplo de utilização da linguagem de programação *R* para a tarefa de classificação por meio do algoritmo *k-NN*
Fonte: Reprodução

Árvores de Decisão

Definições

Alguns algoritmos utilizados nas tarefas de Mineração de Dados são caixas pretas, ou seja, não permitem a interpretação da forma pela qual os resultados foram obtidos. Nesse contexto, a justificativa das decisões pode ser necessária em algumas aplicações.

Há modelos interpretáveis que são resultado de alguns algoritmos, como, por exemplo, as árvores de características ou de decisão.

As árvores de decisão são algoritmos que permitem particionar as características (atributos) de maneira hierárquica. As árvores de decisão são consideradas para as tarefas de classificação, enquanto as tarefas de regressão contam com um modelo similar, denominado árvore de regressão.

Conheça mais detalhes sobre árvores de decisão no vídeo a seguir.
Disponível em: https://youtu.be/xSna88sBWHU

Nesses modelos, os atributos alvo (rótulo dos dados) são identificados por meio da divisão do conjunto de registros a partir dos valores dos atributos preditivos. Considere a Figura 5 como exemplo, veja que, a partir do valor de um atributo preditivo, o conjunto de registros é dividido em duas partes. Ainda nesse exemplo, o grupo que está à direita são todos referentes ao mesmo atributo alvo. Por outro lado, o grupo à esquerda apresenta atributos alvo diferentes. Assim, um novo atributo preditivo é selecionado para dividir, novamente, os registros de acordo com seus respectivos valores. Após essa última divisão, observe que cada grupo resultante apresenta um mesmo atributo alvo.

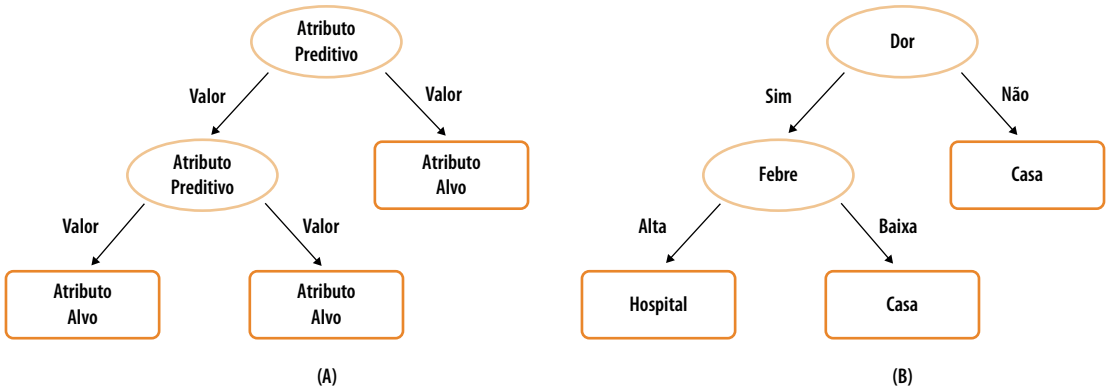


Figura 5 – Exemplo de árvore de decisão
Fonte: Adaptada de ROSSI, 2021

Para ilustrar, de forma mais aplicada, o exemplo anterior, considere a Figura 5B. O atributo preditivo que foi considerado para a primeira divisão dos dados foi o atributo "dor", cujos valores são "sim" ou "não". Veja que os registros cujo valor para esse atributo é "não" são todos classificados com o atributo alvo "casa", ou seja, as pessoas que não apresentaram dor ficaram se tratando em casa. Outro grupo de registros apresenta como atributo alvo tanto o valor "hospital" quanto o valor "casa", portanto não podem ser discri-

minados. Assim, é necessária a seleção de um novo atributo preditivo para uma nova divisão, no caso o atributo febre.

O conjunto de registros foi então dividido em duas partes, a partir dos valores do atributo preditivo "febre". O primeiro é composto por registros cujos valores para o atributo "febre" são todos iguais a "alta" e o outro grupo concentra os registros cujos valores para o atributo preditivo "febre" são todos iguais a "baixa". Veja que, em cada grupo, há uma homogeneidade para o atributo alvo, ou seja no primeiro grupo o valor de todos os atributos alvo é igual a "hospital" e o valor "casa" é observado para o segundo.

O modelo descrito no exemplo é derivado de um conjunto de registros médicos, para os quais temos os sintomas (dor e febre) e o local do tratamento (hospital ou casa). Nesse sentido, podemos utilizar esse modelo para determinar o local de tratamento de um novo paciente, a partir de seus sintomas. Suponha que o paciente não tenha dor então, de acordo com o modelo, ele pode ser tratado em casa. Por outro lado, se o paciente apresentar dor e febre alta deverá ser tratado no hospital. Se a febre for baixa, o paciente pode ser tratado, também, em casa.

A seguir é apresentado um algoritmo básico considerado para a obtenção de uma árvore de decisão.

Algoritmo 2: Árvore de Decisão

Entrada:

X_{tr} : conjunto de treinamento;

L : lista de atributos descritos de um objeto em X_{tr}.

Saída:

Uma estrutura de dados hierárquica organizada como um classificador.

Etapas:

1. Criar um nó N;
2. Se para todo $\vec{x}_i \in X_{tr}$ o rótulo y_i é o mesmo:
3. Retorne N como um nó folha rotulado com y_i.
4. Se L é uma lista vazia

   Retorne N como um nó folha rotulado com y_i mais frequente em X_{tr};

   Senão:

   $A \leftarrow$ primeiro atributo descritivo em L;

   Associe A ao nó N;

   $L \leftarrow L - A$;

   Para cada valor distinto v de A, considerando todo $\vec{x}_i \in X_{tr}$;

   $X_{trv} \leftarrow$ subconjunto de objetos em que $A = v$;

   Se X_{trv} é vazio:

   Retorne N como nó folha rotulado com y_i mais frequente em X_{trn}.

   Senão:

   Associe a N uma subárvore retornada por uma chamada recursiva ao algoritmo, com os parâmetros de entrada: X_{trv} e L;

Retorne N.

Fonte: Adaptado de SILVA *et al.*, 2017

Considere os dados da Tabela 2 para a implementação do algoritmo de árvore de decisão, implementado na linguagem de programação R. Os dados da tabela são referentes à qualidade de serviço em um restaurante. Os atributos descritos deste conjunto de dados são: EP (experiência profissional), QR (qualidade das refeições) e LE (localização do estabelecimento). Cada registro apresenta um rótulo (R) referente ao resultado da operação do estabelecimento.

Tabela 2 – Conjunto de dados de exemplo para o algoritmo Árvore de Decisão

	x_{i1}	x_{i2}	x_{i3}	y_i
	EP	QR	LE	R
$\vec{x}_1$	MUITA	BOA	RUIM	Lucro
$\vec{x}_2$	MUITA	BOA	BOA	Lucro
$\vec{x}_3$	POUCA	BOA	BOA	Lucro
$\vec{x}_4$	POUCA	ÓTIMA	RUIM	Prejuízo
$\vec{x}_5$	POUCA	ÓTIMA	BOA	Prejuízo
$\vec{x}_6$	MUITA	ÓTIMA	BOA	Lucro
$\vec{x}_7$	MUITA	ÓTIMA	RUIM	Prejuízo
$\vec{x}_8$	POUCA	BOA	RUIM	Prejuízo

Fonte: Adaptada de SILVA *et al.*, 2017

Aplicações com a Linguagem *R*

O *RStudio* disponibiliza uma série de funções para a construção de árvores de decisão. Para esta tarefa, vamos utilizar o pacote ***rpart*** (*Recursive Partitioning and Regression Trees*), que contém uma função, de mesmo nome, para a construção de árvores de decisão. A função `rpart()` recebe os seguintes parâmetros de entrada:

```
modelo_ad <- rpart(fórmula, dados, método, controle, parâmetros),
```

em que:

- **`modelo_ad:`** é a variável que armazenará a árvore de decisão;
- **`rpart:`** é a função responsável pela construção da árvore de decisão;
- **`fórmula:`** define qual é o atributo de classe e quais são os atributos descritivos;
- **`dados:`** é um conjunto de dados de entrada;
- **`método:`** é o parâmetro que define se a árvore resultante será de decisão (`class`) ou de regressão (`anova`);
- **`controle:`** é a variável que armazenará os parâmetros de crescimento da árvore;
- **`parâmetros:`** é a variável que define o critério de seleção dos atributos.

O primeiro passo para a obtenção de uma árvore de decisão é o carregamento dos dados no *RStudio*. O procedimento deve seguir de acordo com as descrições feitas para a tarefa de classificação, utilizando um arquivo CSV com os dados de entrada. Veja, na Figura 6, os detalhes do arquivo importado.

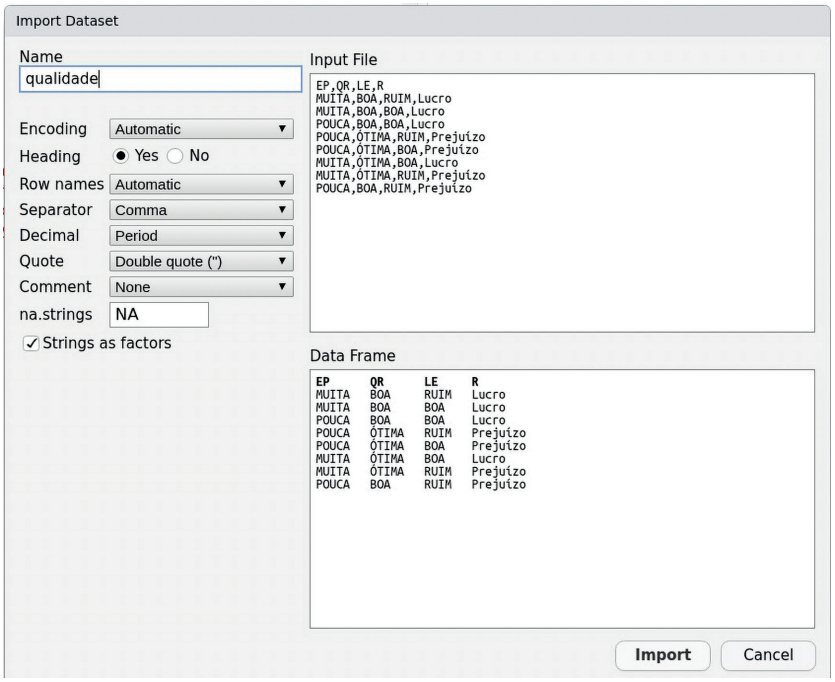


Figura 6 – Resultado da importação de um arquivo CSV (*Input File*) para o *RStudio*, transformando os dados em um *Data Frame*
Fonte: Reprodução

Após a importação dos dados de entrada, a utilização da função `rpart()` deve ser feita da seguinte maneira:

```
modelo_ad <- rpart(R ~ EP + QR + LE, data = qualidade, method = "class",
control = rpart.control(minsplit = 1), parms = list(split = "Information")).
```

O modelo resultante terá o seguinte conteúdo:

```
n= 8

node), split, n, loss, yval, (yprob)

      * denotes terminal node

 1) root 8 4 Lucro (0.5000000 0.5000000)

   2) EP=MUITA 4 1 Lucro (0.7500000 0.2500000)

     4) QR=BOA 2 0 Lucro (1.0000000 0.0000000) *

     5) QR=ÓTIMA 2 1 Lucro (0.5000000 0.5000000)

      10) LE=BOA 1 0 Lucro (1.0000000 0.0000000) *

      11) LE=RUIM 1 0 Prejuízo (0.0000000 1.0000000) *
```

```
3) EP=POUCA 4 1 Prejuízo (0.2500000 0.7500000)
  6) QR=BOA 2 1 Lucro (0.5000000 0.5000000)
   12) LE=BOA 1 0 Lucro (1.0000000 0.0000000) *
   13) LE=RUIM 1 0 Prejuízo (0.0000000 1.0000000) *
  7) QR=ÓTIMA 2 0 Prejuízo (0.0000000 1.0000000) *
```

As linhas que descrevem o modelo são indentadas de acordo com a representação dos níveis da árvore de decisão. Cada linha descreve o número do nó, o atributo e o valor utilizados para a separação, dois valores que indicam o número de exemplares chegando no nó e quantos não correspondem à classe majoritária, a própria classe majoritária e a probabilidade de cada classe, respectivamente.

Por exemplo, os nós 2 e 3 dividem os exemplares do conjunto de dados a partir do atributo `EP`. No nó 2, são concentrados os exemplares cujo valor para o atributo `EP` é `MUITA`, esse subconjunto é formado por quatro exemplares, sendo um deles diferente da classe majoritária `Lucro`. Note que os nós identificados com um asterisco são nós folhas.

A partir da obtenção da árvore de decisão é possível utilizar esse modelo para predizer a classe de um objeto não rotulado. Para isso, o *RStudio* disponibiliza a função `predict()` cuja sintaxe de uso é descrita a seguir:

```
y_estimado <- predict(modelo_ad, exemplares_teste, tipo)
```

onde:

- **`y_estimado:`** é a variável que contém os rótulos dos exemplares de teste;
- **`modelo_ad:`** é o modelo resultante da aplicação da função `rpart()`;
- **`exemplares_teste:`** são os exemplares não rotulados;
- **`tipo:`** define o formato de apresentação dos resultados, pode ser `vector`, `class` ou `prob`.

Considere os seguintes exemplares não rotulados:

EP, QR, LE

MUITA, ÓTIMA, RUIM

POUCA, BOA, BOA

A aplicação da função `predict()` é feita da seguinte forma:

```
y_estimado <- predict(modelo_ad,qualidade_teste,"class"),
```

para esses exemplares, os rótulos preditos foram `Prejuízo` e `Lucro`, respectivamente.

Podemos, ainda, criar uma representação gráfica do modelo gerado. Essa representação é obtida por meio da função `rpart.plot()`. Essa função recebe o modelo da árvore de decisão (gerado pela função `rpart()`) e um valor entre zero e quatro que define o tipo da representação. A chamada para essa função pode ser feita da seguinte forma:

```
plot <- rpart.plot(modelo_ad, type = 3).
```

A representação resultante é apresentada na Figura 7.

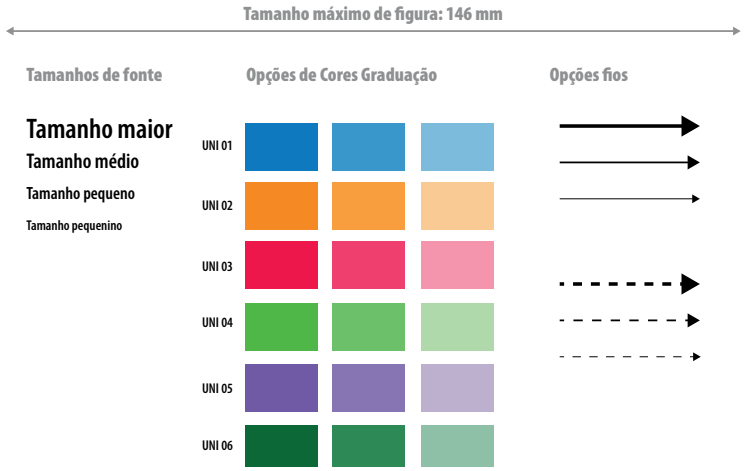


Figura 7 – Representação gráfica da árvore de decisão gerada a partir dos dados de qualidade em serviço de restaurante

O Algoritmo *Naive Bayes*

Definições

Os problemas considerados no âmbito da Mineração de Dados apresentam, de maneira geral, a incerteza como característica comum. Quando utilizamos um classificador, temos como objetivo atribuir uma classe a um objeto, com o maior grau de certeza possível. Nesses casos, por mais que o classificador seja efetivo, há sempre uma possibilidade de erro, ou seja, estamos lidando com cenários incertos.

Os algoritmos discutidos são úteis para a extração de um modelo, baseado no conhecimento que está implícito no conjunto de dados. Por outro lado, a estatística pode ser considerada com o mesmo propósito dos algoritmos, podemos utilizar métodos estatísticos para obter um modelo, padrão ou regra que seja útil na predição de características incertas.

A busca por lidar com as incertezas pode considerar diferentes teorias. Quando temos uma informação probabilista, podemos lidar com ela considerando as teorias de probabilidade ou da evidência. Um contexto de informação imprecisa pode ser trabalhado por meio das teorias dos conjuntos nebulosos (*fuzzy*) ou de aproximação.

Um exemplo de incerteza pode ser observado no contexto do diagnóstico médico. Não é possível afirmar que para todo paciente que tenha como sintoma a febre, a doença que o acomete é a gripe. Para que essa afirmação fosse verdadeira seria necessário a inclusão de outras causas possíveis para o sintoma, como, por exemplo, pneumonia, virose, dentre outras.

Outra tentativa de se obter uma relação de certeza entre doença e sintoma é a partir da doença predizer o sintoma. A afirmação de que todo paciente que tem gripe como doença terá febre como sintoma também é incorreta. Nem todo paciente que tem gripe terá febre. Mesmo que fosse possível listar todas as doenças associadas a um determinado sintoma e vice-versa, a utilização dessas regras seria pouco útil, visto que seriam

demasiadamente longas. Além disso, não há um conhecimento completo sobre o domínio da medicina, assim, é muito provável que haja doenças ou sintomas que não são conhecidos. Porém, supondo o amplo conhecimento sobre o domínio, ainda haveria muita insegurança na aplicação de uma determinada regra a um paciente, visto que, do ponto de vista prático, não é possível realizar todos os testes previstos.

Apesar da evidente impossibilidade de se realizar afirmações assertivas em cenários de incerteza, é possível considerar algum grau de crença nas afirmações descritas anteriormente. Nesse contexto, a teoria da probabilidade fornece os meios para definir esses graus de crença, representados por valores percentuais ou fatores, de modo que se possa estabelecer as chances de ocorrência de determinados fenômenos.

Os fatores de probabilidade variam entre zero, que representa uma crença inequívoca de que a afirmação, ou relação entre afirmações, é falsa, e um, que indica uma crença inequívoca de que a afirmação é verdadeira. Os valores intermediários indicam graus de crença na veracidade da afirmação.

Quando temos novas evidências sobre uma determinada afirmação, a probabilidade sobre ela pode ser alterada. Assim, chamamos de probabilidade a priori ou incondicional aquela definida inicialmente, sem que novas evidências tenham sido obtidas. A probabilidade posterior ou condicional é aquela definida após a obtenção das evidências.

As decisões envolvem determinadas preferências sobre diversos resultados possíveis. Assim, a teoria da utilidade busca representar e analisar as preferências, supondo que todo estado tem um grau de utilidade, então a preferência será dada àquele com maior grau. Nesse contexto, a teoria da decisão une os conceitos das teorias da probabilidade e da utilidade.

A base para a realização de inferência probabilística são os conjuntos de dados a partir dos quais se pode extrair a distribuição de probabilidade conjunta total. Considere a Tabela 4, que é composta por três variáveis lógicas (binárias). Nessa tabela, temos a representação da ocorrência de "dor de dente" e de "cárie", a terceira variável "extração" corresponde à solução para o caso (rótulo). O acento til (~), que precede as variáveis, indica a negação da variável.

Para cada combinação de resultados na Tabela 3, temos a probabilidade de ocorrência, por exemplo, se o paciente tem cárie e dor de dente, a probabilidade de ele precisar extrair o dente é de 0,108 (10,8%) e de não extrair é de 0,012 (1,2%). A soma de todos os fatores de probabilidade na tabela é igual a um, ou seja, temos uma distribuição das probabilidades conjuntas representada na Tabela.

Tabela 3 – Exemplo de distribuição conjunta de probabilidade

	Dor de dente		~Dor de dente	
	extração	~extração	extração	~extração
cárie	0.108	0.012	0.072	0.008
~cárie	0.016	0.064	0.144	0.576

Fonte: Adaptada de LORENA, 2017

A partir da distribuição conjunta das probabilidades, podemos identificar a probabilidade de eventos individuais, como, por exemplo, a probabilidade de se ter cárie, que é expressa como $P(cárie)$ e é obtida pela soma das probabilidades desse evento: $P(cárie) = 0,108 + 0,012 + 0,072 + 0,008 = 0,2$, ou seja, a probabilidade de se ter cárie é de 20%.

Podemos, também, estabelecer a probabilidade de eventos de forma combinada, como a probabilidade de se ter cárie e dor de dente. A notação para esse caso é $P(cárie \vee dor\,de\,dente)$, veja que $\vee$ é o operador lógica da disjunção. Nesse caso, basta que seja feita a soma dos eventos atômicos nos quais a proposição é verdadeira: $P(cárie \vee dor\,de\,dente) = 0,108 + 0,012 + 0,072 + 0,008 + 0,016 + 0,064 = 0,28.$

As probabilidades condicionais podem ser calculadas a partir das probabilidades não condicionais. Nesses casos, temos que realizar a transformação da seguinte maneira:

$$P(a|b) = \frac{P(a \wedge b)}{P(b)}$$

Podemos ler a equação anterior como a probabilidade do evento "*a*" ocorrer; dado que o evento "*b*" ocorreu, é igual à probabilidade de ocorrência dos eventos "*a*" e "*b*" dividida pela probabilidade de ocorrência do evento "*b*". Note que a probabilidade do lado esquerdo da equação é condicional, enquanto as probabilidades do lado esquerdo são não condicionais.

Vejamos um exemplo, qual a probabilidade de uma pessoa ter cárie, dado que ela tem dor de dente?

$$P(cárie|dor\,de\,dente) = \frac{P(cárie \wedge dor\,de\,dente)}{P(dor\,de\,dente)}$$

$$P(cárie|dor\,de\,dente) = \frac{0,108 + 0,012}{0,108 + 0,012 + 0,016 + 0,064} = 0,6$$

Assim, a probabilidade de uma pessoa ter cárie, dado que ela tem dor de dente é de 60%.

A distribuição conjunta, representada na forma de tabela, não apresenta praticidade na construção de sistemas de raciocínio. À medida que o número de variáveis aumenta, a construção dessas tabelas fica inviável, devido ao seu tamanho.

A regra de Bayes é utilizada para a descrição da probabilidade de ocorrência de eventos dependentes, ou seja, aqueles que estão baseados em um conhecimento *a priori*. Essa regra é a base para a realização da inferência bayesiana que é útil para, por exemplo, a predição de classes de objetos.

A regra de Bayes é baseada na regra do produto, a qual considera as seguintes equações:

$$P(a \wedge b) = P(a|b)P(b)$$

$$P(a \wedge b) = P(b|a)P(a)$$

A partir dessa regra, podemos derivar a regra de Bayes, igualando as equações:

$$P(b|a) = \frac{P(a|b)P(b)}{P(a)}$$

Uma aplicação prática para a regra de Bayes é a filtragem de *spam*. A denominação *spam* é aplicada àquelas mensagens eletrônicas que são enviadas sem a solicitação ou autorização do destinatário. Comumente, esse tipo de mensagem tem por objetivo a realização de *marketing* eletrônico, porém, pode-se observar a utilização de *spam* para a aplicação de golpes dos mais variados tipos. A seguir é apresentado o algoritmo *Naive Bayes*.

Conheça mais detalhes sobre o algoritmo *Naive Bayes* no vídeo a seguir.
Disponível em: https://youtu.be/02L2Uv9pdDA

Algoritmo: ***Naive Bayes***
Entrada:
X_{tr}: conjunto de treinamento;
X_{te}: um exemplar de teste não rotulado.
Saída:
y_{te}: rótulo estimado para o exemplar de teste.
Etapas:

1. Calcular a probabilidade para cada classe $K\left(P\left(c_k\right)\right)$;
2. Para cada classe K:
   Inicializar uma variável ($prod$) com 1;
   Para cada atributo descritivo $j \in \vec{x}_{te}$:
   Recuperar os exemplares $\vec{x}_i \in X_{tr}$;
   Organizar os exemplares de acordo com y_i;
   Calcular $P(x_{ij} \mid c_k)$ multiplicando o resultado por $prod$;
   Fazer $P\left(c_k|\vec{x}_{te}\right) = P\left(c_k\right) * prod$.
3. Encontrar $P\left(c_k|\vec{x}_{te}\right)$ que maximiza $P\left(c_k|\vec{x}_{te}\right)$ e retornar como rótulo de $\vec{x}_{te}$.

Fonte: Adaptado de SILVA *et al.*, 2017

Os dados apresentados na Tabela 4 descrevem uma previsão de vendas de um determinado prato em função de características climáticas e serão considerados para a implementação do algoritmo *Naive Bayes* no *RStudio*.

Tabela 4 – Conjunto de dados de exemplo para o algoritmo *Naive Bayes*

	x_{i1}	x_{i2}	x_{i3}	x_{i4}	y_i
	Previsão	Temperatura	Humidade	Vento	Vender
$\vec{x}_1$	Chuva	Frio	Normal	Sim	Parmegiana
$\vec{x}_2$	Chuva	Moderado	Alta	Sim	Parmegiana
$\vec{x}_3$	Sol	Quente	Alta	Não	Parmegiana
$\vec{x}_4$	Sol	Quente	Alta	Sim	Parmegiana
$\vec{x}_5$	Sol	Moderado	Alta	Não	Parmegiana
$\vec{x}_6$	Nublado	Frio	Normal	Sim	Feijoada
$\vec{x}_7$	Nublado	Quente	Alta	Não	Feijoada
$\vec{x}_8$	Nublado	Quente	Normal	Não	Feijoada
$\vec{x}_9$	Nublado	Moderado	Alta	Sim	Feijoada
$\vec{x}_{10}$	Chuva	Frio	Normal	Não	Feijoada
$\vec{x}_{11}$	Chuva	Moderado	Alta	Não	Feijoada
$\vec{x}_{12}$	Chuva	Moderado	Normal	Não	Feijoada
$\vec{x}_{13}$	Sol	Frio	Normal	Não	Feijoada
$\vec{x}_{14}$	Sol	Moderado	Normal	Sim	Feijoada

Fonte: Adaptada de SILVA *et al.*, 2017

Aplicações com a Linguagem *R*

Seguindo o mesmo procedimento descrito nas tarefas anteriores, os dados em formato CSV devem ser importados para o *RStudio*. O resultado dessa importação é apresentado na Figura 8.

O pacote "e1071" disponibiliza a função `naiveBayes()` para a instalação do pacote no *RStudio*. Digite na linha de comando o seguinte: `install.packages("e1071")`. Após a instalação, selecione o pacote na aba *Packages*.

A sintaxe da função `naivebayes()` tem a seguinte forma:

```
modelo_naiveBayes <- naiveBayes(dados, rótulo, laplace),
```

em que:

- **`modelo_naiveBayes:`** é uma variável que receberá o modelo;
- **`naiveBayes():`** é a função que implementa o algoritmo;

- **dados:** são os dados de entrada;
- **rótulo:** é um arranjo com as classes de cada exemplar;
- **laplace:** é um número real que controla a suavização de Laplace.

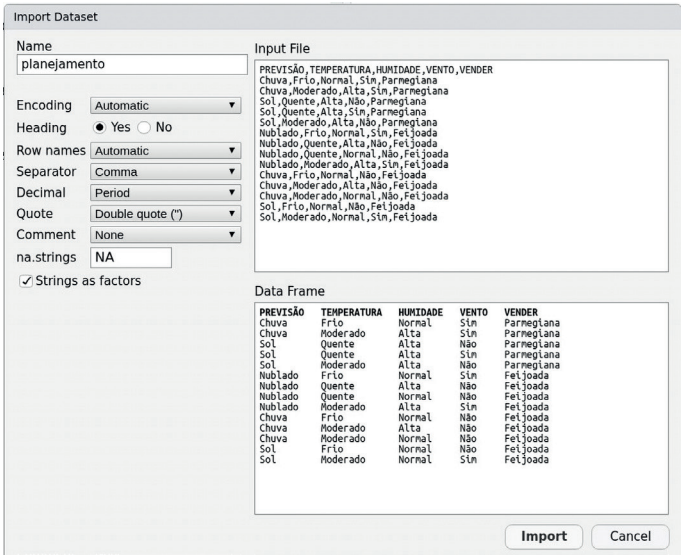


Figura 8 – Resultado da importação de um arquivo CSV (*Input File*) para o *RStudio*, transformando os dados em um *Data Frame*
Fonte: Reprodução

Para a obtenção do arranjo com as classes, podemos utilizar o seguinte comando:

```
classes <- planejamento[1:14,5:5].
```

Desse modo, a utilização da função com os respectivos parâmetros tem o seguinte formato:

```
modelo_naiveBayes <- naiveBayes(planejamento,classes,0).
```

Com a obtenção do modelo, podemos utilizá-lo para predizer a classe de um objeto não rotulado. Vamos criar o seguinte objeto que será rotulado pelo classificador:

```
exemplar_teste <- data.frame( PREVISÃO = “Sol”, TEMPERATURA = “Frio”, HUMIDADE = “Normal”, VENTO = “Sim” ).
```

Esse novo exemplar será rotulado com a utilização da função `predict()`, já descrita anteriormente, e com a passagem do modelo como parâmetro, da seguinte forma:

```
p <- predict( modelo_naiveBayes, exemplar_teste, type = “class” ).
```

Desse modo, o objeto `p` contém a classe do novo exemplar que, para esse exemplo, será `Feijoada`.

Em Síntese

Nesta Unidade, conhecemos três importantes algoritmos para tarefas preditivas; classificação, árvore de decisão e *Naive Bayes*. Além disso, conhecemos as funções, em linguagem de programação *R*, que implementam esses algoritmos. Nas próximas unidades, continuaremos nossa exploração a respeito das tarefas de Aprendizagem de Máquina com aplicações em Mineração de Dados por meio do *RStudio*.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Leitura

e1071: ***Misc Functions of the Department of Statistics, Probability Theory Group (Formerly: E1071), TU Wien***
https://bityl.co/BEVH

Functions for Classification
https://bityl.co/BEVL

Plot 'rpart' Models: An Enhanced Version of 'plot.rpart'
https://bityl.co/BEV1

rpart: Recursive Partitioning and Regression Trees
https://bityl.co/BESo

Referências

BECKER, J. L. **Estatística básica:** transformando dados em informação. Porto Alegre Bookman 2015. (*e-book*)

CASTRO, L. N.; FERRARI, D. G. **Introdução à mineração de dados:** conceitos básicos, algoritmos e aplicações. São Paulo: Saraiva, 2016.

SILVA, L. A.; PERES, S. M.; BOSCARIOLI, C. **Introdução à mineração de dados:** com aplicações em R. Elsevier Brasil, 2017.

LIMA, M. F. W. P. **Introdução a *Big Data* e *Internet* das Coisas**. Porto Alegre: SAGAH, 2018.

LORENA, A. C., FARIA, F. A. **Representação do Conhecimento** – Lidando com incerteza. Notas de aula. UNIFESP (São Paulo), 2015.

ROSSI, L. **Técnicas para converter dados em conhecimento**. São Paulo: Universidade Cruzeiro do Sul, 2021.


Cruzeiro do Sul
Educacional